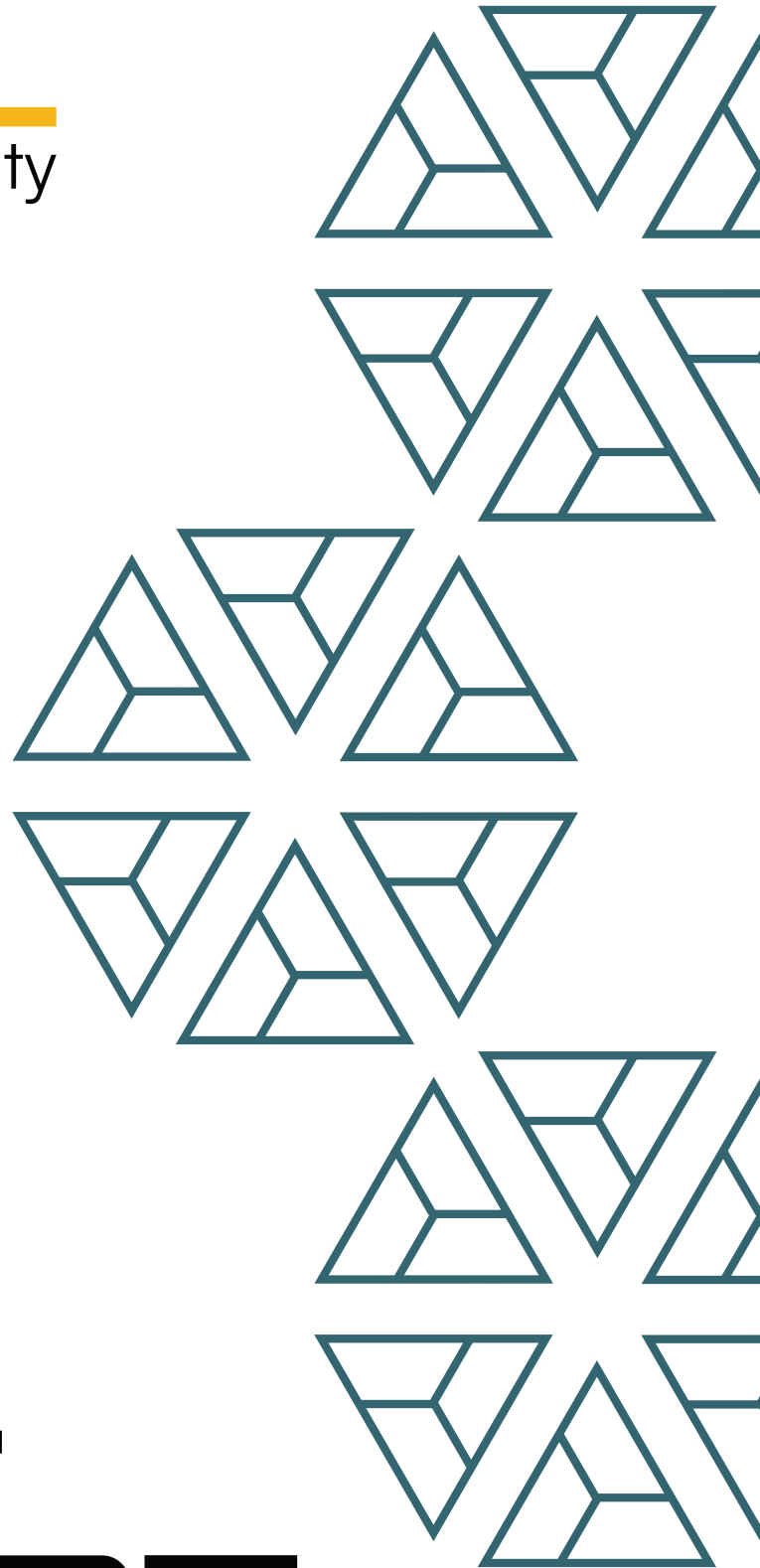




BAIL
security



Lista Dao
Fixed Term

FINAL REPORT

May '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – Fixed Term- Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	3					
Medium	14					
Low	20					
Informational	30					
Governance	6					
Total	73					

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

BrokerMath

The `BrokerMath` contract implements helper functions for the `LendingBroker` contract.

It implements an oracle function to scale the collateral price based on open interest used by the `Moolah` vault. Besides this, it implements functions to update the accounting during loan issuance, repayments, and liquidations, along with share-asset conversions.

Appendix: peek

The `peek` function implements the collateral oracle used by the `Moolah` contract. This function first checks the value of the collateral via an oracle and then scales down its price according to the accrued debt, following the expression below. Pricing is done in terms of the loan token, which is either a stablecoin or wrapped BNB.

$$> \text{collateralPrice} = \text{oraclePrice} - \text{AccruedDebt} / \text{TotalCollateral}$$

Issue_01	Loan-token price drift expands minLoan in token units, autonomously stranding existing broker positions
Severity	High
Description	<code>Moolah.minLoan</code> computes the per-token threshold as $\text{minLoanValue} * 10^{\text{decimals}} / \text{price}$, where <code>price</code> is read live from the loan-token oracle. <code>BrokerMath.checkPositionsMeetsMinLoan</code> calls <code>Moolah.minLoan</code> on every state-changing user flow and on liquidation. Because price is read at call time, the threshold is not static: any drop in the loan-token price expands minLoan in token units, and positions that were comfortably above the threshold at open time can fall into the band autonomously with no governance action. Once a position is in the band, every borrow, repay, convert, refinance, and liquidation cascade for that user reverts via <code>_validatePositions</code>. Liquidators cannot incrementally clear the user either, since the cascade's post-deduct validator runs into the same trip, and broker interest continues to accrue while no one can act. This was not a meaningful concern when the broker only supported stable loan tokens, in which case <code>BrokerMath.peek</code> returned a hardcoded loan-token price. The loan-token price branch in <code>BrokerMath.peek</code> now reads from the oracle directly, and <code>LendingBroker</code> has been extended with native-BNB support. Together, these changes signal that the broker is now expected to operate with volatile loan tokens, in particular <code>WBNB</code>.
Recommendations	Consider implementing functionality like repaying multiple or all broker positions at once so users can escape the deadlock.
Comments / Resolution	

Issue_02	Principal of a dynamic position getting set to 0 even when interest is non-zero allows the user to bypass interest repayment
Severity	Medium
Description	During liquidation, the principal of dynamic position can drop to 0 even when there is non-zero interest remaining in case $\{principalToDeduct \geq dynamicPosition's\ principal \ \&\& \ interestToDeduct < dynamicPosition's\ interest\}$ This will allow the user to bypass the repayment of this interest since the bounded underlying collateral that necessitated repayment can now be moved by repaying the principals of fixed positions. For example: <i>collateral in moolah = 100</i> <i>debt in moolah = 80</i> <i>dynamic debt in broker = 40</i> <i>fixed debt in broker = 40</i> <i>total debt in broker = 80</i> <i>total interest in broker = 8</i> <i>interest in dynamic position = 6</i> <i>interest in fixed position = 2</i> now if 40 (ie. 50%) is liquidated, <i>interestToDeduct = 4</i> <i>principalToDeduct = 40</i> <i>dynamic position's principal = 40 - 40 = 0</i> <i>dynamic position's interest = 6 - 4 = 2</i> If the user repays the fixed position, then <i>debtInMoolah</i> will become 0, and hence the user can withdraw their collateral freely without

	ever repaying the dynamic position's interest of 2. The orphan state is also reachable as a tiny dust orphan from floor rounding alone in any partial or full clear of a dynamic position when the rate is above 1.0. The cascade decrements normalizedDebt separately for the interest leg and the principal leg, each rounded down. Because floor rounding is non-additive, the sum of the two decrements falls short of the original normalizedDebt by 1-2 wei. The principal hits zero, but a wei or two of normalizedDebt remains. This dust path does not require a fixed position to coexist, a single-dynamic-position partial clear is enough to produce it. This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Re-work the liquidation mechanism; proportional reduction of interest and principal is flawed.
Comments / Resolution	

Issue_03	Positions can be left stranded if minLoanValue is modified
Severity	Medium
Description	The <code>checkPositionsMeetsMinLoan</code> function iterates over each loan position and makes sure each of them is larger than the minimum <code>Moolah</code> loan amount. The issue is that if multiple positions are below this limit, the user is stranded. This is because no borrows or repays will be possible, since every function calls the <code>_validatePositions</code> function in the end. Thus, if there are 2 such borrow positions with their loans below the minimum limit, repaying one of them will fail due to the presence of the other, and the user will be completely stuck. This can happen if the <code>Manager</code> role of the <code>Moolah</code> vault changes the <code>minLoanValue</code> or if the oracle price changes. If they raise this value such that multiple existing loans fall below this, the loan owners will be unable to repay their positions. This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Consider adding functionality that allows repaying multiple or all broker positions at once.
Comments / Resolution	

Issue_04	Array index in <code>posIdToRemove</code> defaults to zero when position ID is not found
Severity	Low
Description	In <code>refinanceMaturedFixedPositions</code>, if a position ID is not found, it stores that index as the default value 0 in the <code>posIdToRemove</code> array since the iteration is skipped. <i>if (position.posId == 0) continue;</i> This would cause unnecessary iteration[s] in the second for loop that removes position IDs. For example, position IDs passed by bot are [100, 101, 102], but 101 does not exist in user's fixed positions. Hence, <code>posIdToRemove</code> evaluates to [100, 0, 102]. This causes unnecessary gas wastage for the bot.
Recommendations	Consider acknowledging this behaviour.
Comments / Resolution	

Issue_05	Move <code>userFixedPositions</code> outside the for loop to save gas
Severity	Informational
Description	In <code>refinanceMaturedFixedPositions</code>, we can move <code>broker.userFixedPositions[user]</code> outside the for loop since user fixed positions remain the same on each iteration. Additionally, <code>broker.userFixedPositions[user]</code> is retrieved twice in the function itself, which is unnecessary.
Recommendations	Cache the <code>userFixedPositions</code> array outside the for loop.
Comments / Resolution	

Issue_06	Return early in <code>checkPositionsMeetsMinLoan</code> if the dynamic position does not meet <code>minLoan</code> requirement
Severity	Informational
Description	In the function <code>checkPositionsMeetsMinLoan</code>, we can return early if <code>isValid</code> was set to false due to the dynamic position not meeting <code>minLoan</code> requirement. Currently, even if <code>isValid</code> is already false, we loop through all fixed positions, which would cause unnecessary gas costs.. <pre> if (dynamicDebt > 0 && dynamicDebt < minLoan) { isValid = false; } // check fixed positions for (uint256 i = 0; i < fixedPositions.length; i++) { FixedLoanPosition memory _fixedPos = fixedPositions[i]; uint256 fixedPosDebt = _fixedPos.principal - _fixedPos.principalRepaid; if (fixedPosDebt > 0 && fixedPosDebt < minLoan) { isValid = false; } } </pre>
Recommendations	Consider returning early if <code>isValid</code> is already set to false.
Comments / Resolution	

Issue_07	<code>principal > principalRepaid</code> filter is unreachable under current invariants
Severity	Informational
Description	<code>BrokerMath.sortAndFilterFixedPositions</code> filters input positions with <code>if [p.principal > p.principalRepaid]</code> . Every mutation site for <code>fixedLoanPositions[user]</code> either pushes a new position with <code>principalRepaid = 0</code> , removes the entry on <code>principalRepaid >= principal</code> , or wipes the array entirely. By induction, every entry in storage satisfies <code>principal > principalRepaid</code> , so the filter never excludes anything in practice.
Recommendations	Either remove the filter (slight gas saving, clearer intent) or keep it as a defensive sanity guard.
Comments / Resolution	

BrokerInterestRelayer

The **BrokerInterestRelayer** contract supplies loan tokens to the **Moolah** vault, which are then given out as loans to **LendingBroker** users. This contract's main purpose is to use the protocol-generated interest revenue to provide liquidity to the vault.

Privileged Functions

- `supplyToVault`
- `addBroker`
- `removeBroker`

Core Invariants:

INV 1: Only authorized broker contracts can call this contract to supply liquidity

INV 2: Only add liquidity if the contract has more than `minLoan` tokens

Issue_08	Brokered interest flushed to the curated vault can be sandwiched
Severity	Medium
Description	The relayer's flush calls <code>Moolah.supply</code> with the broker-market params, the entire balance as assets, and the curated vault as the on-behalf address. This updates Moolah's per-market ledger only; no code path inside the vault runs, so the count of vault shares held by depositors does not change. But <code>MoolahVault.totalAssets</code> iterates the vault's <code>withdrawQueue</code> and sums each enabled market's <code>expectedSupplyAssets</code> live, so as long as the broker market is in the <code>withdrawQueue</code> the vault's NAV jumps by exactly the flushed amount in a single block while share count stays flat. Consider the following situation: Vault pre-state: 1,000,000 LISUSD of value with the broker market in the <code>withdrawQueue</code>. A 50,000 LISUSD brokered interest flush is about to land. An attacker frontruns with a deposit of 9,000,000 LISUSD at the current 1.0 share price. The broker's repay tx pushes 50,000 through the relayer. NAV jumps from 10,000,000 to 10,050,000 in the same block. The attacker back-runs with a redeem, receiving $9,000,000 * 10,050,000 / 10,000,000 = 9,045,000$. Attacker pockets 45,000 [~90% of the flush]. Long-term depositors are diluted to 5,000 of their fair share.
Recommendations	Consider smoothing <code>totalAssets</code> via time-weighted reveal of brokered interest. It has to be noted that the MoolahVault in itself is not part of the auditing scope.
Comments / Resolution	

Issue_09	<code>supplyToVault</code> can fail due to a faulty minimum loan check
Severity	Medium
Description	The <code>supplyToVault</code> function checks if the balance in the contract is higher than the <code>minLoan</code> amount, and if so, transfers it to the <code>Moolah</code> vault. <pre>uint256 minLoan = MOOLAH.minLoan(MOOLAH.idToMarketParams(IBrokerBase(msg.sender).MARKET_ID())); if (balance >= minLoan) {</pre> The issue is that <code>Moolah</code> implements this check differently. In <code>Moolah</code>, first, the supply shares are calculated by rounding down the assets. <pre>if (assets > 0) shares = assets.toSharesDown(market[id].totalSupplyAssets, market[id].totalSupplyShares);</pre> Then, in <code>_checkSupplyAssets</code>, the supply assets are calculated by rounding down the shares. <pre>return uint256(position[id][account].supplyShares).toAssetsDown(market[id].totalSupplyAssets, market[id].totalSupplyShares) >= minLoan(marketParams);</pre> So there are two rounding downs involved, so the <code>Moolah</code> check is more stringent than the <code>Relayer</code> check, and requires a higher amount of assets to pass the <code>minLoan</code> check. Thus, the relayer check might pass, but the <code>Moolah</code> check might fail if the amount in question is just the right value. In this case, the whole transaction might fail. This can stop repayment and liquidation transactions, which is a major issue. This issue was previously identified in an earlier audit of the

	contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Consider wrapping the <code>MOOLAH.supply</code> call in a try-catch block to prevent it from reverting the transactions if not successful. Also, adjust the <code>minLoan</code> check so that a deposit is attempted only if the balance is some threshold amount above the <code>minLoan</code> amount. Another option would be to simulate the deposit transaction and then check if the assets controlled in Moolah are above the <code>minLoan</code> amount.
Comments / Resolution	

Issue_10	No way to specify <code>marketId</code> for collected revenue
Severity	Low
Description	The same relayer contract can be used by multiple brokers to supply loan tokens to the Moolah vaults. The relayer picks the market to supply to by calling back the <code>msg.sender</code>. <pre><i>MOOLAH.supply(MOOLAH.idToMarketParams(IBrokerBase(msg.sender).MARKET_ID[]), balance, 0, vault, "");</i></pre> The issue is that it sends any accumulated funds to the last calling market. If the contract does not contain enough funds, it will keep accumulating tokens in the contract until it reaches <code>minLoan</code>. So, revenue generated from market A will be accumulated until a call from market B forces all that revenue into market B. Thus, the revenue generated by a broker/market will not flow to the same market. This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Consider keeping track of the accumulated revenue on a per-broker basis.
Comments / Resolution	

Issue_11	Brokered interest accumulated in the relayer below Moolah's <code>minLoan</code> threshold has no exit path other than future broker activity
Severity	Low
Description	The relayer's flush is the only outflow. It transfers the broker's interest payment in, then supplies to the vault only when the relayer's balance is at least <code>minLoan</code> . If the balance stays below the threshold, the function returns silently, and the residual sits until a future broker flush pushes it across. The contract has no skim, no admin rescue, no broker refund, and no setter for the vault address, so outflow depends entirely on continuous broker activity plus a stable <code>minLoan</code> , neither of which is guaranteed: the manager can remove brokers or raise <code>minLoanValue()</code> , borrower activity can stop, and a loan-token price change expands <code>minLoan()</code> in token units automatically.
Recommendations	Add a permissionless <code>flushResidual</code> function that picks any registered broker's market as the destination and supplies whenever the balance is greater than zero, plus an admin-restricted <code>rescueTokens</code> for the no-brokers-registered case.
Comments / Resolution	

Issue_12	<code>BrokerInterestRelayer</code> uses string-based require reverts instead of custom errors
Severity	Informational
Description	The relayer reverts with string messages on every guard, while the rest of the broker stack uses custom errors. Custom errors are cheaper at deploy and revert time, give callers a stable four-byte selector to match against, and keep error handling uniform across the codebase.
Recommendations	Replace each string-based require in <code>BrokerInterestRelayer</code> with a custom error
Comments / Resolution	

Issue_13	Store <code>MARKET_ID</code> to memory to save gas
Severity	Informational
Description	The <code>MARKET_ID</code> is retrieved twice from the broker in the function <code>supplyToVault</code>. <pre><i>IBrokerBase[msg.sender].MARKET_ID()</i></pre> Retrieving it twice leads to unnecessary gas wastage. Instead, the <code>MARKET_ID</code> can be retrieved once, stored in memory, and loaded in both instances to save gas.
Recommendations	Consider caching the value in memory and using the memory variable in both instances.
Comments / Resolution	

Issue_14	Possible out-of-gas error in <code>getBrokers</code>
Severity	Informational
Description	The function <code>getBrokers</code> can throw an out-of-gas error if many brokers are registered over time. If this function is utilized by an external contract in the future, this can cause unintended reverts.
Recommendations	Consider acknowledging this risk and monitoring the number of brokers added.
Comments / Resolution	

LendingBroker

The **LendingBroker** contract is the main contract in the protocol that handles the loans. There are 2 types of loans offered by the system: dynamic loans, with dynamic interest rates, and fixed-rate loans with fixed rates and loan durations.

The contract interacts with an exclusive **Moolah** vault, where only the broker is allowed to borrow from and repay to. This special vault is also set up with 0 interest rates, and the actual interest calculations are handled by the **LendingBroker** instead.

When a user borrows funds, the contract borrows from the **Moolah** vault on the user's behalf and thus requires to be whitelisted by the user on **Moolah**. During repayments, the interest is deducted and kept by the broker, while the principal is repaid back to the **Moolah** vault under the same user's account.

Loans can either be taken out as a stablecoin or as wrapped BNB.

Liquidations are carried out by calling the **liquidate** function. The current **Liquidator** and **PublicLiquidator** contracts do not yet support this functionality since it is still under development and out of scope for this audit.

Appendix: Dynamic loans

Dynamic loans apply a variable interest rate to the position. When users borrow funds, their debt is denominated in shares according to the current **getRate** value. The rate then keeps increasing over time, requiring higher repayments to nullify the same borrowed shares.

The interest accrual is handled in the **RateCalculator** contract. The Bot sets a rate per second via the **setRatePerSecond** function, and the **getRate** value keeps increasing based on this value. The **accrueRate** function updates the current interest state of the contract.

During repayments, priority is given to the interest amount, and only after clearing the interest can the actual principal be repaid.

Appendix: Fixed-term loans

Fixed-term loans are loans with a constant interest rate for a constant loan duration. Unlike dynamic loans, where the interest rate can be changed by the Bot anytime, fixed-term loans record the interest rate at the very beginning and use simple interest calculation to calculate the interest owed based on that amount.

Users can own multiple fixed-rate loan positions up to a specified **maxFixedLoanPositions** limit. Whenever repayments are made, the contract records the repaid interest and repaid principal in separate variables. If a user repays the loan before the end of the term, they attract an extra penalty which is equal to half of the remaining interest.

Appendix: Interest handling

Interest is handled differently for the different loan types.

For dynamic loans, the loans are denominated in borrow shares according to the current interest rate value, which keeps going up over time. During repayment, the shares to be paid back are calculated according to the interest rate value at that moment. The contract deducts the borrow shares based on the repaid value and also keeps track of the principal amount separately. When the entire principal is repaid, the position is deleted.

For fixed-term loans, the interest is calculated using the fixed interest rate. Interest repayments are recorded in an `interestRepaid` variable, and if any of the principal is also repaid, a penalty is added on top.

Appendix: Loan conversions

Dynamic loans can be converted to fixed-rate loans via the `convertDynamicToFixed` function. This nullifies the user's borrow shares and instead opens a fresh fixed-term loan of the principal + interest amount.

Appendix: Liquidation

The liquidation mechanism has been heavily modified to accommodate the `Broker` contract charging interest instead of `Moolah`.

Firstly, the `Broker` implements an oracle for the collateral token. Unlike normal `Moolah` vaults, where the collateral is priced with an oracle directly, here the collateral price is actually scaled down based on the accrued interest amount according to the following formula.

> `collateralPrice = oraclePrice - AccruedDebt/TotalCollateral`

By scaling down the collateral price, the `Moolah` vault can now effectively compare the collateral with the loan, including the interest.

Once a position is liquidatable, whitelisted liquidators can call the contract and pay off part of the principal and accrued interest and seize the user's collateral tokens. After liquidation, the dynamic and fixed-term positions are adjusted to reflect the new liquidated state. The dynamic position is adjusted first, repaying its collateral and interest. Then the fixed-term positions are adjusted one by one, from the loan with the earliest end time to the latest.

Appendix: Economic simulator

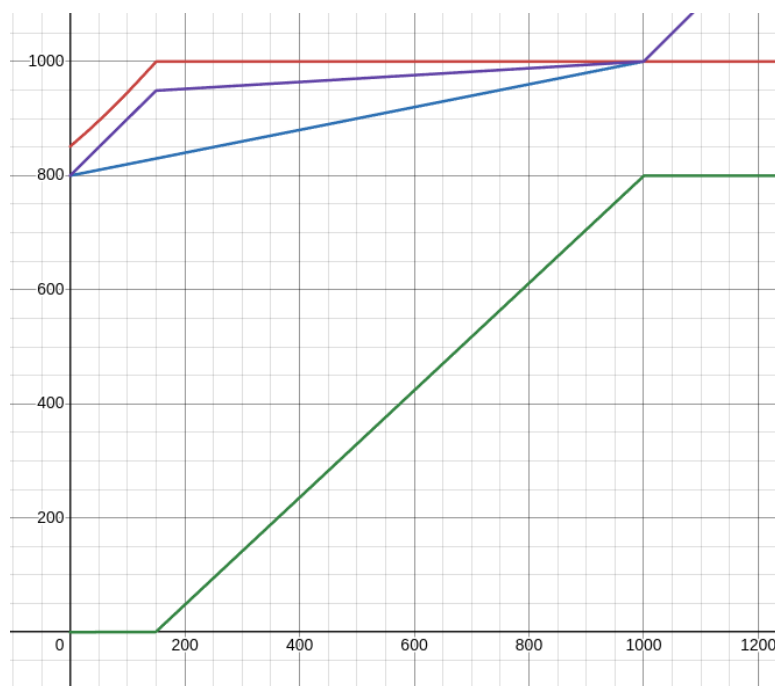
A simulator was set up to see the behaviour of the system on Desmos. The initial conditions chosen were of both stablecoins as collateral and loan, and an ltv of 0.8 for the market, which leads to an incentive factor of 1.0638. The graph can be found [here](#). This was used to investigate a number of issues.

In the graph,

- The X-axis is the amount of interest accrued.
- p_{collat} is the price of collateral. Given by $(1-x/x_{\text{collat}})$, capped to 0.
- $i_{\text{incentive}}$ is the incentive factor of the market
- L_{loan} is the loan amount. This can be adjusted to check various scenarios.
- D_{debtMax} (Blue line) is the total debt to the system. This includes the Moolah borrow $(x_{\text{collat}} \times p_{\text{collat}} \times 0.8)$ and the interest (x) .
- A_{seized} (Red line) is the total amount seized on a liquidation. This is given to the liquidator.
- A_{paid} (Purple line) is the total amount paid by the liquidator, including interest. This scenario always goes for the maximum liquidation amount, including bad debt scenarios.
- D_{badDebt} (Green line) is the amount of bad debt incurred in the system.

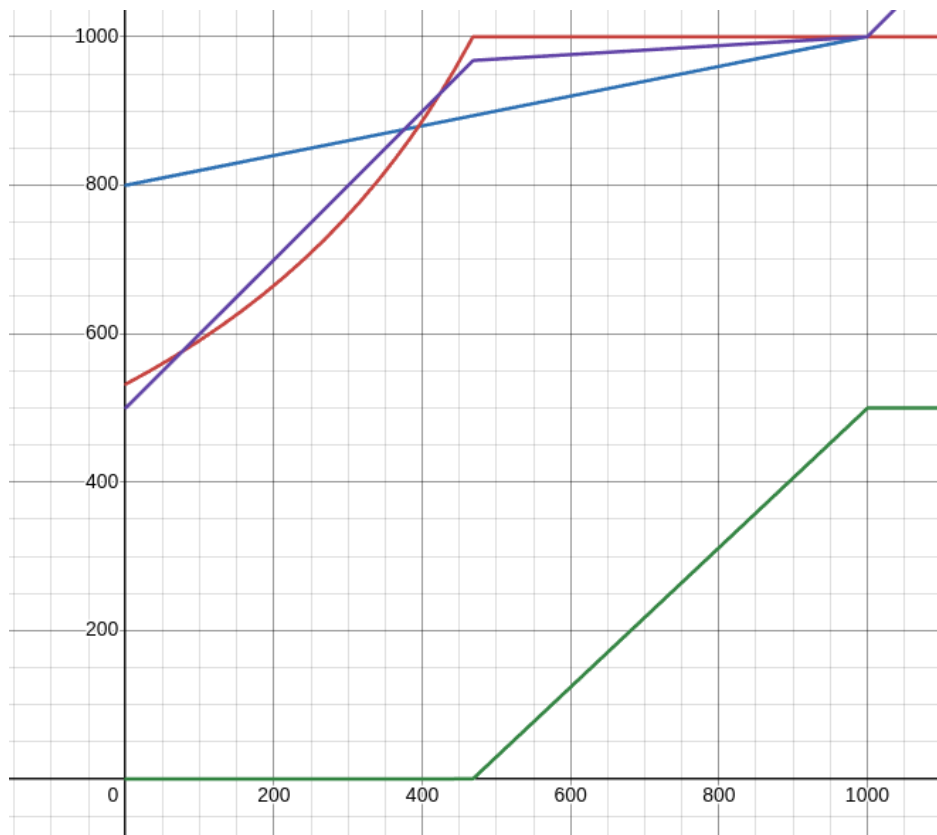
The mathematical expressions of the various quantities can be found in the [hyperlink](#).

The following is a snapshot when there's a loan of 800 tokens.



The Blue line (D_{debtMax}) increases as the X-axis progresses (interest). This shows that even though Moolah only allows an ltv of 0.8, since here you can have debt as interest, the system has an ltv which increases as the interest accrues. Liquidations can occur at any point above this blue line, not below it. The Red line A_{seized} is the amount of collateral assets seized. This reaches the maximum collateral of 1000 and then stays fixed. The purple line A_{paid} is the amount of principal+interest paid by the liquidator. If this line is below the red line [seized > paid], it means liquidations are profitable. The green line shows the bad debt, which only starts after the entire collateral is exhausted.

Some issues can be identified by changing the loan size to 500.



Here, one can observe a triangular slice bounded by the blue, purple, and red lines around the point (400,900). This is above the blue line, thus liquidation is possible here. But the red line is below the purple line, so more needs to be paid to do the liquidation than is seized, and is thus unprofitable. Also note how, as the loan amount is decreased, the first possible liquidation point (where the purple line goes above the blue line) shifts to the right. For a loan of 1 token, this intersection is very close to 1000 interest, forcing bad debt very easily onto the system.

Privileged Functions

- onMoolahLiquidate
- refinanceMaturedFixedPositions
- setMarketId
- updateFixedTermAndRate
- setMaxFixedLoanPositions
- setBorrowPaused
- pause
- unpause
- toggleLiquidationWhitelist

Core Invariants:

- INV 1: Sum of loan principals must reflect the total amount borrowed from Moolah.
- INV 2: During repayments, the interest amount is always repaid before the principal.
- INV 3: Each loan position must be of at least minLoan value
- INV 4: The oracle and BrokerMath must always return the price with 8 decimals
- INV 5: IRateCalculator external calls are only ever related to dynamic positions
- INV 6: Any paid interest must be supplied to MoolahVault
- INV 7: Normalization logic is only ever related to dynamic positions
- INV 8: Fixed Position: position.interestRepaid must be reset on each principal repay
- INV 9: Whenever a fixed position is fully repaid, it must enter _removeFixedPositionByPosId
- INV 10: A user cannot create more Fixed Loan positions than maxFixedLoanPositions
- INV 11: Partial repayment of Fixed Position must reset interest
- INV 12: Partial repayment of Fixed Position must increase position.principalRepaid
- INV 13: Liquidation must first handle Dynamic Position and then Fixed Positions
- INV 14: normalizedDebt must represent the borrowed amount, normalized at the time of the borrowing
- INV 15: denormalizedDebt must represent the accrued interest
- INV 16: position.principalRepaid must never be reset
- INV 17: Top-up borrow of dynamic position must not influence interest
- INV 18: The denormalized amount includes principal and debt
- INV 19: Whenever a new borrow happens for a Dynamic Position, the amount must be normalized
- INV 20: Before any interaction with Dynamic Positions, the rate must be accrued
- INV 21: Repayment of Fixed Position before end must incur penalty
- INV 22: Repayment must first repay interest and then account for (penalty), principal
- INV 23: Any repaid interest must be supplied to MoolahVault
- INV 24: Liquidations should only happen via seizeAssets in full. No partial liquidations, no liquidations via repaidShares

Issue_15	Liquidations permanently fail when <code>collateralToken</code> is <code>StableSwapLpCollateral</code>
Severity	High
Description	The <code>StableSwapLpCollateral</code> contract has the <code>onlyMoolah</code> modifier on the <code>transfer/transferFrom</code> functions. This means only Moolah can transfer these tokens. <pre> function transfer(address to, uint256 value) public override onlyMoolah returns (bool) { address owner = _msgSender(); _transfer(owner, to, value); return true; } /// @dev only Moolah can call transferFrom function transferFrom(address from, address to, uint256 value) public override onlyMoolah returns (bool) { address spender = _msgSender(); _spendAllowance(from, spender, value); _transfer(from, to, value); return true; } </pre> However, during the liquidation process, these tokens are sent to the <code>LendingBroker</code> by Moolah. The broker then attempts to send these tokens to the <code>BrokerLiquidator</code>, where <code>StableSwapLpCollateral</code> is redeemed and swapped to loan tokens. However, since the <code>LendingBroker</code> does not have permission to transfer these tokens, the liquidation flow reverts. <pre> // [3] transfer seized collateral to liquidator IERC20[COLLATERAL_TOKEN].safeTransfer(liquidator, IERC20[COLLATERAL_TOKEN].balanceOf(address(this)) - liquidationContext.preCollateral </pre>
Recommendations	Consider allowing the <code>LendingBroker</code> to transfer <code>StableSwapLpCollateral</code> tokens to the <code>BrokerLiquidator</code> contract.

	Note that this would require upgrading any existing StableSwapLpCollateral contracts deployed, which requires reviewing the upgrade logic thoroughly to avoid any potential issues from arising in out-of-scope contracts.
Comments / Resolution	

Issue_16	Broker liquidation undercharges the liquidator in bad-debt scenarios, gifting the gap as profit
Severity	Medium
Description	In broker markets, Moolah's liquidation math uses a user-adjusted collateral price [market price minus accumulated broker interest spread per unit of collateral] instead of the market price. The broker layers a proportional interest charge on top to recover what the discount removed, but the proportional charge scales with the principal actually repaid in the partial liquidation. In bad-debt scenarios, partial repayments cover only a small slice of total debt, so the proportional add-back is small while the discount has already removed everything. Vanilla Moolah, with the same total debt accrued via IRM, charges against the full market price and recovers the full LIF margin against the seized collateral. Consider the following situation: Borrower owes a total of 150 LISUSD against 60 collateral units priced at \$1 each. LLTV 0.8, $LIF = 1 / (1 - 0.3 * 0.2) = 1.0638$. Position is unhealthy ($150 > 48$ max borrow) and in a bad-debt regime ($150 > 60$ collateral). Liquidator submits the <code>seizedAssets</code> path for the full 60 collateral. In vanilla Moolah, the 150 of debt sits in <code>totalBorrowAssets</code> via IRM accrual (the user borrowed 100 originally, IRM grew the debt to 150). <code>collateralPrice</code> = \$1 [market]. <code>seizedAssetsQuoted</code> = 60. <code>repaidAssets</code> = $60 / 1.0638 = 56.40$. After liquidation, the user has 0 collateral and a remaining debt of 93.60. Bad-debt branch fires: <code>totalBorrowAssets</code> and <code>totalSupplyAssets</code> each decrease by 93.60, so suppliers absorb 93.60. Liquidator paid 56.40 for \$60 of collateral, profit 3.60. In a broker market, the 150 splits as 100 principal at Moolah (broker market runs 0% IRM, so <code>totalBorrowAssets</code> = 100 for this user) and 50 broker interest tracked separately at the broker. <code>collateralPrice</code> in liquidate is the user-adjusted price: $market - (broker_interest /$

	collateral] = $1 - 50/60 = 0.167$. seizedAssetsQuoted = $60 * 0.167 = 10.02$. repaidAssets = $10.02 / 1.0638 = 9.42$. The broker callback computes $\text{interestToBroker} = \text{mulDivCeiling}[9.42, 150, 100] - 9.42 = 4.71$. Liquidator pays $9.42 + 4.71 = 14.13$ for \$60 of collateral, profit 45.87. After liquidation, Moolah's bad-debt branch fires for the remaining Moolah-side debt: $100 - 9.42 = 90.58$ socialized to suppliers. The broker's residual $50 - 4.71 = 45.29$ of interest sits as an uncollectable claim. Combined protocol loss: vanilla 93.60 (suppliers only) vs broker $90.58 + 45.29 = 135.87$ (suppliers + broker). The 42.27 LISUSD gap matches the liquidator's extra profit ($45.87 - 3.60 = 42.27$). The mechanism is structural: the discount removes $(\text{broker_interest} / \text{collateral}) * \text{seized} = 50$ of the effective seizure value, but the proportional add-back recovers only $\text{principal_repaid} * \text{broker_interest} / \text{principal} = 4.71$. In a bad-debt regime where principal_repaid is small (driven down by the discount itself), the add-back cannot catch the discount. The defect does not appear in healthy liquidations: when total debt is below the collateral-value/LIF threshold, principal_repaid is large enough that the proportional add-back fully recovers the broker interest, and the liquidator pays approximately what they would in vanilla Moolah. It only appears in bad-debt liquidations, where it grows unbounded as broker interest approaches collateral value. It has to be noted that there are generally speaking multiple issues with the pricing logic which can result in overpaying or underpaying by the liquidator.
Recommendations	Consider using the market price for the liquidation incentive math instead of the discounted price, and rely on the broker's proportional fee as the sole mechanism for recovering broker interest from the liquidator. The discount's intended role of accelerating the unhealthy signal is already covered by the separate health check that uses market price plus full broker debt; it does not need to also reduce the liquidator's required payment.

	Please note that such a change will require a re-audit of the control-flow and comes with additional risk.
Comments / Resolution	

Issue_17	Unpaid interest on partial liquidations
Severity	Medium
Description	When a position gets partially liquidated, Inside <code>deductFixedPositionDebt</code> function when there is some interest left to pay (<code>repayInterestAmt < accruedInterest</code>) and <code>repayPrincipalAmt > 0</code>, the remaining unpaid interest (<code>accruedInterest - repayInterestAmt</code>) will be skipped due to resetting the repayments variables: <pre> if (repayPrincipalAmt > 0) { [...] // reset repaid interest to zero (all accrued interest has been cleared) p.interestRepaid = 0; // reset repaid time to now p.lastRepaidTime = block.timestamp; } </pre> As these two variables are reset, it will start counting interest from 0, and the remaining interest that was left will not be paid. As the comment states, “<i>(all accrued interest has been cleared)</i>”, this will be true for repayments, but in partial liquidations, it may not be the case; sometimes, not all interest is cleared before resetting these variables. <u>Scenario:</u> Fixed position: <code>principal=100</code>, <code>accruedInterest=10</code> Partial liquidation pays: 5 interest, 50 principal In <code>deductFixedPositionDebt</code>: <code>repayInterestAmt = 5</code>, <code>repayPrincipalAmt = 50</code> - First if: <code>p.interestRepaid += 5</code> - Second if: <code>p.principalRepaid += 50</code> <code>p.interestRepaid = 0</code> ← RESET (was 5)

	- <code>p.lastRepaidTime</code> = now ← RESET Result: - Position NOT deleted ($50 < 100$) - 5 interest was paid - 5 interest was ERASED - Protocol loses 5 tokens This flow works on <code>repay</code> function because it makes sure that the whole interest is paid first. But here, due to partial liquidations, before resetting the repayment variables, the contract doesn't make sure that all interest is paid, therefore the remaining interest is left unpaid. <i>Note: There is a similar issue that describes that interest gets zeroed out when all the principal is paid and the left interest to pay > 0. But the root cause of this issue is the reset of the repayments variables when interest left to pay > 0 and <code>repayPrincipalAmt</code> > 0, but don't get fully repaid.</i> This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	This fix is non-trivial. We recommend always executing full liquidations only.
Comments / Resolution	

Issue_18	<code>convertDynamicToFixed</code> silently wipes orphan dynamic interest after partial liquidation
Severity	Medium
Description	The broker tracks a dynamic loan as <code>principal</code> plus <code>normalizedDebt</code>, with the spread representing accumulated interest. Normal repay decrements both fields together. <code>convertDynamicToFixed</code> breaks the coupling: when called against a position whose principal is already zero, it deletes the entire dynamic struct from storage, including any residual interest balance, without payment. The principal-zero state is reachable through partial liquidation because the cascade allocates the proportional broker-interest fee against the dynamic position first, partially clearing dynamic interest while the principal payment fully clears dynamic principal, leaving principal of zero with residual interest still in storage. Consider the following situation: User holds a dynamic position with 100 principal plus 200 interest, plus a fresh fixed F1 with 1000 principal and 0 interest. A liquidator partial seizure clears the dynamic principal but only a fraction of the dynamic interest (the proportional fee budget runs out). Dynamic state post-liquidation: principal 0, residual interest 181. User calls <code>convertDynamicToFixed()</code> with the <code>minLoan()</code> of 10. The broker borrows 10 fresh from Moolah and supplies it to the vault as revenue. The dynamic position's debt is decremented by 10, but the principal is still 0. The <code>delete</code> branch fires. The entire dynamic struct is zeroed, including the 181 residual. A new fixed F2 of 10 is created. User cost: 10 plus a small fixed-term interest. Protocol loss: 171.
Recommendations	Gate the dynamic-position deletion on both principal and remaining interest being zero, or reject convert-to-fixed when there is no principal to move. Alternatively, The simplest and non intrusive fix is always executing full liquidations only.

Comments / Resolution	
-----------------------	--

Issue_19	Liquidation cascade can leave a fixed position's principal in the dead zone between zero and <code>minLoan</code> , reverting the entire liquidation
Severity	Medium
Description	The broker enforces an invariant that every position's principal must be either zero or at least Moolah's <code>minLoan()</code> threshold. User-facing flows (<code>borrow</code>, <code>repay</code>, <code>convert</code>) check this on every state-changing operation, so the dead zone is unreachable through normal user activity. The liquidation cascade does not respect it. Each position takes a deduction up to whatever budget remains. If the deduction lands the remaining principal strictly between zero and <code>minLoan</code>, the cascade keeps going. The post-cascade validator iterates every position and reverts the entire liquidation if any sits in the dead zone. A borrower can also deliberately engineer this state. Calling <code>convertDynamicToFixed(minLoan, termId)</code> repeatedly while the position is healthy fragments dynamic principal into multiple fixed positions sized at exactly <code>minLoan</code>. When the position later becomes unhealthy, the cascade's clean-input space shrinks to integer multiples of <code>minLoan</code>, and any partial-liquidation budget that doesn't align with those multiples lands a fragment in the band and reverts. The convert path bypasses health checks when the dynamic position has no accrued interest at the moment of the call (<code>interestToRepay</code> clamps to zero, skipping the inner <code>Moolah.borrow()</code>), so even an unhealthy borrower with a freshly-borrowed dynamic position can fragment without first repaying. The same outcome is reachable by directly opening multiple fixed positions of exactly <code>minLoan()</code> via <code>broker.borrow()</code>. Similarly, liquidations can be frontrun via repayments such that the partial liquidation leads to a revert due to <code>_validatePositions()</code>.

	Consider the following situation: minLoan threshold is 10 LISUSD. User holds F1 with 100 remaining principal and F2 with 500 remaining principal. After dynamic deduction completes, 95 of the principal budget remains for the fixed cascade. Cascade applies the full 95 to F1, leaving F1 remaining at 5 (dead zone). F2 is untouched. Validator iterates: $F1 = 5 < 10$, reverts the entire liquidation even though F2's principal would be enough to cover the principal needed to keep F1 above <code>minLoan()</code>.
Recommendations	Make the cascade aware of the threshold: for each position, only allow two outcomes, fully consume the principal or leave the remaining principal at exactly <code>minLoan</code>. If the available budget would land strictly between, take only down to <code>minLoan</code> and carry the unused budget forward to the next position. Alternatively, remove <code>_validatePositions</code> in <code>onMoolahLiquidate</code>. However, this requires the fix for "Positions can be left stranded if <code>minLoanValue</code> is modified" to be implemented.
Comments / Resolution	

Issue_20	Liquidations/repayments concerns during paused state
Severity	Medium
Description	When the broker is paused, interest continues to accrue on all positions. Since liquidations and repayments are also blocked during this state, it can result in protocol bad debt, and previously healthy positions may become liquidatable by the time the market is unpaused. This can cause a sudden wave of unexpected liquidations immediately after unpausing. This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Consider at least allowing repayments and liquidations during the paused state.
Comments / Resolution	

Issue_21	Liquidations allow for unpaid interest to be zeroed out
Severity	Medium
Description	This occurs because in the function <code>onMoolahLiquidate</code>, after deducting from the dynamic position, any leftover value to be deducted is split into <code>interestLeftover</code> and <code>principalLeftover</code>. Next, the function will use these values to deduct value from the fixed positions. In the case where <code>principalLeftover</code> is enough to deduct all the principal of a fixed position, but not enough to zero out/eliminate all the interest, then the position will erroneously be deleted and the interest due with it. This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	The fix is non-trivial; consider ensuring that the interest is paid off before deleting a position. For example, by only allowing full liquidations.
Comments / Resolution	

Issue_22	Regions of unprofitable liquidations
Severity	Medium
Description	Normally, in Moolah vaults, liquidations are always profitable, thus always incentivized. Even if there isn't enough collateral, only a part of the debt needs to be paid back to seize the entire collateral, and the rest becomes bad debt. This is important to ensure liquidators are active, protecting the health of the protocol. However, here, due to how the collateral price is scaled, there are pockets where liquidations are straight-up unprofitable. For example, consider the following situation: Collateral of 1000 tokens, loan = 500 tokens, lltv = 0.8 Interest accrues to 374 tokens. $\text{deltaDebt} = 374$, collateral price = $1 - 0.374 = 0.626$ $\text{max allowed debt} = 1000 \times 0.626 \times 0.8 = 500.8$. The loan isn't liquidatable yet, since the moolah debt (500) is less than the max 500.8. If the interest accrues to 376 tokens, it becomes liquidatable. $\text{deltaDebt} = 376$, collateral price = $1 - 0.376 = 0.624$ $\text{max allowed debt} = 1000 \times 0.624 \times 0.8 = 499.2$. Since $500 > 499.2$, this can be liquidated. For liquidation, the incentive factor is 1.0638 for 0.8 lltv. So the user pays off the debt of 500 tokens, and receives collateral of $500 \times 1.0638 / 0.624 = 852.4$ tokens. However, the liquidator also needs to pay off the 376 tokens of interest, thus a total of 876 tokens, while they get only 852.4 tokens of collateral. Thus, this is unprofitable. This becomes profitable only when interest accrues to 425 tokens. Then,

	deltaDebt = 425, collateral price = $1 - 0.425 = 0.575$ paying off 500 loan gives $500 \times 1.0638 / 0.575 = 925.04$ tokens. This is used to pay off the 500 loan + 425 interest with a 0.4 profit. Thus, liquidations are not always profitable. This is also true if the interest ever accrues to be more than the collateral itself, in which case the system prioritizes paying off the interest first, so liquidations are always unprofitable then. Profitable liquidations are necessary to keep the system healthy. This issue is also described graphically in Appendix: Simulator. In the context of BrokerLiquidator, this may result in failed flash liquidations as the swaps would not provide sufficient loan tokens for repayment. The BrokerLiquidator contract does not allow unprofitable flash liquidations, since the collateral must cover the loan. Thus only the normal liquidation mode functions can be used, and the Bot role must eat the loss when liquidating such positions. Please note that we deem the general calculation math as not fully correct as it can result based on the state in over or undercharging for the liquidator. This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Re-work the liquidation method. The broker should forego interest to keep the liquidations profitable if required.
Comments / Resolution	

Issue_23	Non-divisible share counts in broker liquidations drift the market off its clean accounting ratio
Severity	Medium
Description	Broker markets at Moolah maintain a clean invariant that <code>totalSupplyShares</code> equals <code>totalSupplyAssets</code> times the virtual-share offset, because the broker market runs at zero Moolah-side IRM and the ratio is locked across the market's lifetime. The share-driven liquidation path breaks it: the liquidator passes a number of shares to burn directly, the protocol derives the asset deduction by ceiling division, and when the share count is not a clean multiple of the offset, the asset side rounds up by one wei more than the divisible case. Shares drop by exactly the amount specified, but assets drop by one wei extra. Each non-divisible call adds drift, shared by every user in the market. Consider the following situation: A borrower has a clean 100 LISUSD position when a price drop makes them liquidatable. A liquidator submits the share-driven path with a non-divisible share count. Protocol burns the shares but reduces assets by one wei extra. The market is now drifted off its clean ratio. Price recovers. The borrower calls <code>repay</code> with their full debt amount. Repay's round-down conversion produces a share count one short of the borrower's stored share balance. Burn leaves one share behind, worth a fraction of a wei in asset terms, below <code>minLoan</code>. Repay reverts with <code>REMAIN_BORROW_TOO_LOW</code>. Three failure modes share the same drift. In one, the conversion overshoots the user's stored share balance, the subtraction underflows, and every borrower whose stored shares are still on the clean multiple hits this on full repayment (one liquidation against one user can lock full repayment for every clean-state borrower in the market). In another, the conversion undershoots and leaves a

	residual below <code>minLoan</code>, blocking repayment indefinitely. In the third, new borrowers in a drifted market are evaluated incoherently between the two layers: the broker's <code>checkPositionsMeetsMinLoan</code> compares raw principal to <code>minLoan</code>, while Moolah's <code>_checkBorrowAssets</code> uses <code>borrowShares.toAssetsDown</code> which loses the accumulated drift to floor rounding. A borrow at exactly <code>minLoan</code> passes the broker's check but trips Moolah's <code>REMAIN_BORROW_TOO_LOW</code>, requiring the user to supply slightly above <code>minLoan</code> even though the broker considers the position valid. Once the market has drifted, new borrowers created post-drift inherit the off-ratio mapping at <code>Moolah</code> while the broker still records <code>dyn.principal</code> cleanly at the asset amount. The two views diverge starting at the moment of borrow, and the proportional fee formula in <code>onMoolahLiquidate</code> (<code>mulDivCeiling(repaidAssets, totalDebtAtBroker, debtAtMoolah)</code>) uses the drifted <code>Moolah</code> view as the divisor while the broker view is the numerator. Each subsequent non-divisible liquidation adds more drift, compounding the divergence across all users in the market.
Recommendations	Consider rejecting non-divisible share counts during liquidation. This can be done by requiring the <code>repaidShares</code> input to be a multiple of <code>1_000_000</code>: <pre>[shares = 1_000_000 % 0]</pre>
Comments / Resolution	

Issue_24	Incorrect penalty calculation during repayments
Severity	Low
Description	During repayments, a penalty amount is calculated if a fixed-term loan is closed before the expiry. <pre> penalty = _getPenaltyForFixedPosition(position, UtilsLib.min[repayPrincipalAmt, remainingPrincipal]); // supply penalty into vault as revenue if (penalty > 0) { repayPrincipalAmt -= penalty; _supplyToMoolahVault[penalty]; } </pre> The issue is that the penalty is applied on repayPrincipalAmt amount. However, the actual principal amount being paid off is lower than this, since the penalty is deducted from this itself. Say the principal repayment amount is 100, and the penalty is 5%. According to the code, 5 tokens will be deducted as penalty, and 95 will be used to pay off the principal. But this means a penalty rate of $5/95 = 5.26\%$ was actually applied. This is because the penalty was applied to the penalty amount itself as well. Instead, the repayment should be $100/1.05 = 95.238$ and the penalty should be $0.05 * 100 / 1.05 = 4.762$. In this case, the penalty is 5% of the principal repayment amount.
Recommendations	Consider changing the calculations to make the penalty calculation accurate. Alternatively this issue can be acknowledged if the calculation fits the intended design choice
Comments / Resolution	

Issue_25	<code>_transferLoanToken</code> silently switches to native-BNB delivery when a WBNB provider is registered
Severity	Low
Description	<code>LendingBroker._transferLoanToken</code> reads <code>MOOLAH.providers[MARKET_ID, WBNB]</code> at call time and unwraps WBNB into native BNB whenever the loan token is WBNB and a provider is registered. Provider state is mutable via MANAGER-only <code>setProvider</code> , so the same broker market can flip from ERC20-only to native delivery without any signal to integrators. A contract that has been borrowing successfully starts reverting on the next borrow if it lacks a payable fallback.
Recommendations	Decouple the delivery mode from the provider registration or consider acknowledging this issue.
Comments / Resolution	

Issue_26	<code>LendingBroker._validatePositions</code> is stricter than Moolah's post-liquidation check, blocking partial liquidations that Moolah would accept
Severity	Low
Description	Moolah's <code>_isHealthyAfterLiquidate</code> accepts a position whose remaining debt is below <code>minLoan</code> as long as the position is healthy. <code>LendingBroker._validatePositions</code> delegates to <code>checkPositionsMeetsMinLoan</code> , which strictly requires every individual position's principal to be either zero or at least <code>minLoan</code> with no health-based escape. So, a partial liquidation that lands a fixed or dynamic position in the $[0, \text{minLoan}]$ band passes Moolah's gate but reverts at the broker, aborting the entire liquidation.
Recommendations	Consider aligning the two checks after a liquidation. However, this would need functionality that allows a user to repay multiple broker positions at once. Otherwise, the user's position could get into a state described in other findings where <code>_validatePositions</code> blocks user operations. Alternatively, acknowledge this issue and keep the stricter check.
Comments / Resolution	

Issue_27	Early-repay penalty can exceed the principal being repaid
Severity	Low
Description	<code>LendingBroker.repay</code> [fixed] computes <code>penalty = _getPenaltyForFixedPosition(position, min[repayPrincipalAmt, remainingPrincipal])</code> and then subtracts: <code>repayPrincipalAmt -= penalty</code>. The penalty formula is approximately <code>repayAmt * annual_rate * timeLeft / 2</code>, so the penalty exceeds <code>repayAmt</code> whenever <code>annual_rate * timeLeft > 2</code>. With <code>MAX_FIXED_TERM_APR</code> fixed at 30% and <code>term.duration</code> unbounded above, any term longer than ~6.67 years can produce a penalty greater than the principal portion of the user's repayments, causing the subtraction to underflow.
Recommendations	Cap <code>term.duration</code> at a value consistent with <code>MAX_FIXED_TERM_APR</code> (e.g., 5 years for 30%) so the penalty cannot exceed the principal at any point during the term.
Comments / Resolution	

Issue_28	Sub-minLoan orphan dynamic interest cannot be converted to a fixed loan
Severity	Low
Description	The broker tracks dynamic interest separately from Moolah's <code>borrowShares</code>, so an orphan dynamic position (<code>principal = 0</code>, <code>normalizedDebt > 0</code>) is not subject to <code>checkPositionsMeetsMinLoan</code>, which inspects only principal. The orphan's broker-tracked debt can therefore sit below <code>minLoan</code> without ever triggering validation. However, the <code>convertDynamicToFixed</code> needs to push at least <code>minLoan</code> worth of new fixed-position principal into storage to pass <code>_validatePositions</code>, since the new fixed position's principal is checked against <code>minLoan</code> immediately after creation. When the orphan's actual debt is below <code>minLoan</code>, the convert path creates a fixed position with <code>principal = orphan_debt < minLoan</code>, <code>_validatePositions</code> reverts, and the conversion fails.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	

Issue_29	<code>refinanceMaturedFixedPositions</code> doesn't validate the position
Severity	Informational
Description	As explained in another issue, liquidations can leave one or more positions below <code>minLoan</code>. Suppose any of these positions are refinanced into a dynamic loan that did not previously exist for the user. In that case, the fixed loan below <code>minLoan</code> (due to liquidation) is effectively transferred to the new dynamic loan. The consequences are transferred as well: the user will be forced to repay/borrow first his dynamic position to push it out of <code>minLoan</code> value before managing his fixed loans. This behavior is allowed because the function does not validate the resulting positions after the position's value transfer, which should not be permitted. <code>refinanceMaturedFixedPositions</code> doesn't validate the position This issue was previously identified in an earlier audit of the contract and acknowledged by the team. It is included here again for completeness and tracking purposes.
Recommendations	Consider adding a validation step to ensure that a new dynamic loan cannot be created below the <code>minLoan</code> threshold.
Comments / Resolution	

Issue_30	Fixed position repay's natspec for <code>amount</code> omits the <code>msg.value</code> override note that the dynamic repay documents
Severity	Informational
Description	<code>LendingBroker</code> has two repay overloads, both <code>payable</code> and both treating <code>msg.value</code> as the effective amount when the caller sends native BNB (the function wraps it to WBNB and ignores the amount argument). The dynamic-repay overload documents this in its natspec: " <i>@param amount The amount to repay (overridden by msg.value when sending native BNB)</i> ". The fixed-repay overload uses the same logic but its natspec just says " <i>@param amount The amount to repay</i> ", giving integrators no hint that the argument is silently overridden when value is attached.
Recommendations	Update the fixed-repay overload's <code>@param amount</code> natspec to mirror the dynamic overload.
Comments / Resolution	

Issue_31	User with no active broker positions can perform position conversions to create zero-principal fixed positions
Severity	Informational
Description	Assume a malicious user has no dynamic or fixed positions. This user can pass in amount = 1 wei to <code>convertDynamicToFixed</code>. The function only checks if the <code>amount</code> parameter is not zero. <pre>if (amount == 0) revert ZeroAmount();</pre> When the execution continues, the amount is set to 0 since both <code>interestToRepay</code> and <code>principalToMove</code> clamp down to 0 due to no dynamic position principal and interest existing. <pre>uint256 interestToRepay = UtilsLib.min(amount, totalInterest); uint256 principalToMove = UtilsLib.min(amount - interestToRepay, position.principal); amount = interestToRepay + principalToMove;</pre> The fixed position is then created with the principal set to 0. Function <code>_validatePositions</code> does not revert since <code>checkPositionsMeetsMinLoan</code> skips the <code>minLoan</code> check if the <code>fixedPosDebt</code> is 0. <pre>for (uint256 i = 0; i < fixedPositions.length; i++) { FixedLoanPosition memory _fixedPos = fixedPositions[i]; uint256 fixedPosDebt = _fixedPos.principal - _fixedPos.principalRepaid; if (fixedPosDebt > 0 && fixedPosDebt < minLoan) { isValid = false; } }</pre> The user could intentionally create a maximum number of fixed positions with principal as 0 to cause gas wastage for the BOT on liquidations (assuming this malicious user has either another liquidatable fixed position or a dynamic position in the future).

Recommendations	Consider disallowing the dynamic-to-fixed position conversion if no dynamic position debt exists.
Comments / Resolution	

Issue_32	Protocol does not support transfer-tax and rebasing tokens
Severity	Informational
Description	The LendingBroker and other contracts in scope do not support fee-on-transfer and rebasing tokens. For example: - Alice attempts to borrow 100 tokens from a Moolah market through the broker. - The broker receives these loan tokens however the actual token balance of the contract will be slightly lower due to the transfer-tax applied on the transfer from Moolah to the broker. - The broker attempts to transfer out 100 loan tokens to Alice, however the call reverts since the contract holds 100 - taxed amount of tokens.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	

Issue_33	Possible out-of-gas scenario during liquidations, depending on <code>maxFixedLoanPositions</code> configuration
Severity	Informational
Description	During liquidations, we iterate through the user's fixed positions array in four instances: 1. <code>getTotalDebt</code> - $O(n)$ 2. <code>sortAndFilterFixedPositions</code> - $O(n^2)$ 3. <code>_removeFixedPositionByPosId/_updateFixedPosition</code> - $O(n)$ 4. <code>_validatePositions.</code> - $O(n)$ This can become expensive if the <code>maxFixedLoanPositions</code> configuration is extremely high and can possibly cause liquidations to fail.
Recommendations	Consider keeping control over the <code>maxFixedLoanPositions</code> configuration. A safety mechanism would be to implement an upper cap to it.
Comments / Resolution	

Issue_34	Validate positions before external call
Severity	Informational
Description	In both the repay functions and the dynamic position borrow function, consider validating the positions before performing the external token transfer call. This would ensure the external call cannot manipulate the position state. Currently, there does not seem to be a risk with this, but it is a safer order of operations.
Recommendations	Call <code>_validatePositions</code> before excess token transfer in the repay functions and before the loan token transfer in the dynamic position borrow function.
Comments / Resolution	

Issue_35	<code>LendingBroker.liquidate</code> natspec is outdated and misdescribes both the caller whitelist and the cascade ordering
Severity	Informational
Description	The natspec block above <code>LendingBroker.liquidate()</code> has two stale claims. First, the <code>@notice</code> states "<i>Only Liquidator.sol and PublicLiquidator.sol contracts are allowed to call this function</i>". Second, the <code>@dev</code> says fixed positions are "<i>settling fixed-rate positions sorted by APR and remaining principal</i>", but the actual cascade sort is "sorting by end time ascending", with no use of APR or remaining principal as sort keys.
Recommendations	Update the <code>@notice</code> to reference <code>BrokerLiquidator.sol</code> , and rewrite the <code>@dev</code> to say "settling fixed-rate positions in order of earliest end time first" so it matches the implementation.
Comments / Resolution	

Issue_36	Fixed position removal check is overly broad, given the upstream repayment cap
Severity	Informational
Description	<code>LendingBroker.repay</code> and the cascade's <code>_deductFixedPositionsDebt</code> both use <code>if (p.principalRepaid >= p.principal)</code> to decide between removal and update. Every increment to <code>principalRepaid</code> is bounded by <code>remainingPrincipal = principal - principalRepaid</code> , so <code>principalRepaid + repayAmt <= principal</code> post-update. The <code>>=</code> check is therefore functionally <code>==</code> , never trips on the strict-greater branch.
Recommendations	Either tighten the comparison to <code>==</code> for clarity (signals the invariant explicitly), or keep <code>>=</code> as a sanity guard,
Comments / Resolution	

Issue_37	Fixed position with interest > 0 and principal = 0 can be refinanced to delete interest
Severity	Informational
Description	In the LendingBroker, if any fixed position holds a non-zero interest value and the principal is zero, the fixed position can be refinanced to a dynamic position by the BOT to delete the interest from the fixed position. This occurs because the function refinanceMaturedFixedLoanPositions only updates the dynamic loan position if the principal is non-zero. <pre>// update dynamic position if changed if (refinancedPrincipal > 0) { dynamicLoanPositions[user] = updatedDynPos; emit DynamicLoanPositionBorrowed(user, refinancedPrincipal, updatedDynPos.principal); }</pre> It is currently not possible to achieve this state where interest is non-zero, and principal is zero for a fixed position. However, the issue is being reported to avoid potential issues in the future with contract upgrades/integrations.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	

RateCalculator

The **RateCalculator** contract implements the mechanism to accrue interest for dynamic loans, which has already been described in the **Appendix** in the **LendingBroker** section. In this contract, the Bot role can set an interest accrual rate at any time. Every second, the rate value increases by this amount. When loans are taken out, the borrower's shares are calculated by dividing by this ever-increasing number. Thus, during repayment, a larger amount needs to be repaid to negate the borrowed shares during issuance.

The contract implements an APY, where the rate is raised to the power of the time elapsed, to account for frequent accruals.

Privileged Functions

- `batchSetRatePerSecond`
- `setRatePerSecond`
- `setMaxRatePerSecond`
- `registerBroker`
- `deregisterBroker`

Core Invariants:

INV 1: Rate value must be monotonically increasing

INV 2: Rate must increase by the same percentage point in a duration, irrespective of how many times it is accrued in that duration.

Issue_38	MANAGER role can deregister active broker to brick user operations
Severity	Governance
Description	The MANAGER role has the ability to deregister any broker while it has active debt using the function deregisterBroker. This would lead to failure of all dynamic position functionality i.e., borrow, repay, and liquidate, since it involves interest accrual. Re-registering the same broker resets currentRate to RATE_SCALE while leaving borrower positions intact, silently wiping all accrued broker interest and turning previously-unhealthy positions back into healthy ones.
Recommendations	Consider acknowledging this risk and using a multi-signature wallet for the MANAGER role.
Comments / Resolution	

Issue_39	Remove unused PAUSER_ROLE
Severity	Informational
Description	The RateCalculator contract implements a PAUSER role that is never used. <pre>bytes32 public constant PAUSER = keccak256("PAUSER");</pre>
Recommendations	Consider removing the unused PAUSER constant declared.
Comments / Resolution	

Issue_40	Emit event early in <code>setMaxRatePerSecond</code> to save gas
Severity	Informational
Description	Function <code>setMaxRatePerSecond</code> can emit the <code>MaxRatePerSecondSet</code> event early to avoid the creation of the memory variable <code>oldRate</code>, which is only used in the event emission. <pre> uint256 oldRate = brokers[_broker].maxRatePerSecond; brokers[_broker].maxRatePerSecond = _maxRatePerSecond; emit MaxRatePerSecondSet(_broker, oldRate, _maxRatePerSecond); </pre>
Recommendations	Consider removing the memory variable and emitting the event early as shown: <pre> emit MaxRatePerSecondSet(_broker, brokers[_broker].maxRatePerSecond, _maxRatePerSecond); brokers[_broker].maxRatePerSecond = _maxRatePerSecond; </pre>
Comments / Resolution	

PositionManager

The `PositionManager` contract is an atomic migration helper that moves a user from a variable-rate Moolah market into a fixed-term broker market. It does not maintain borrower debt storage itself. Instead, it uses a Moolah flash loan to repay debt in the source market, withdraws collateral from that source market, supplies the same collateral into the destination market, and then borrows a fixed-term position through the destination market's `LendingBroker` to repay the flash loan.

The contract is designed for “common market to fixed-term market” migration. The source market and destination market must share the same loan token and collateral token. The destination market must have an equal or higher LLTV than the source market. The user must authorize `PositionManager` in Moolah, because the migration repays the user's old debt, withdraws the user's old collateral, supplies collateral into the destination market, and then borrows fixed-term debt on behalf of the user through the destination broker.

The migration can repay the source market either by asset amount or by borrow shares. In asset mode, the user provides `borrowAmount`, and the migration repays that many loan-token assets in the source market. In share mode, the user provides `borrowShares`, and `PositionManager` estimates the flash-loan amount using `MoolahBalancesLib.expectedMarketBalances()` and `SharesMathLib.toAssetsUp()`. Share mode is intended for exact share repayment, especially full migration, because it can clear an exact number of Moolah borrow shares.

Privileged Functions

- `emergencyWithdraw`

Core Invariants:

INV 1: Cannot migrate without withdrawing collateral

INV 2: Can only migrate to market with same collateral and loan token

INV 3: Can only migrate

INV 4: Only `BNBProvider` and `SlisBNBProvider` should be used within migrations

INV 5: `assetsRepaid` must always be equal to `assets`

INV 6: `PositionManager` must be authorized for `onBehalf` in providers

Issue_41	Multiple risks in case of compromised governance roles
Severity	Governance
Description	There are multiple risks the contract currently has if the <code>DEFAULT_ADMIN_ROLE</code> or <code>MANAGER</code> role is compromised: - Privileged addresses can renounce the <code>DEFAULT_ADMIN_ROLE</code> and <code>MANAGER</code> roles to leave the contract in a state where: - Emergency withdrawals cannot be performed - The contract cannot be upgraded <code>DEFAULT_ADMIN_ROLE</code> can upgrade the contract to a malicious implementation. Since the contract is authorized by users to perform Moolah operations on their behalf, user positions can be at risk in case of role compromise. The same risk exists in the <code>LendingBroker</code> contract, where the broker has Moolah-level authority to borrow on behalf of anyone.
Recommendations	Consider acknowledging this risk and using multi-signature wallets to prevent roles from being renounced or malicious operations from being executed instantly.
Comments / Resolution	

Issue_42	Incorrect native token handling in <code>onMoolahFlashLoan</code> causes migration DOS
Severity	Medium
Description	The <code>PositionManager</code> allows users to migrate variable-rate borrow positions to fixed-term positions in a broker. However, migration does not correctly handle the <code>isNative</code> case correctly when the <code>outMarket</code> has a native token provider, but the <code>inMarket</code> does not, and vice versa. For example: - Assume <code>inMarket</code> and <code>outMarket</code> have <code>collateralToken</code> as <code>WBNB</code> - The <code>outMarket</code> has a native token provider (which unwraps collateral and returns native <code>BNB</code>) - The <code>inMarket</code> does not have a provider (moolah only accepts ERC20 <code>WBNB</code> [else branch on Line 164]) - In this scenario, the <code>supplyCollateral</code> operation fails since the contract holds native <code>BNB</code> while Moolah expects <code>WBNB</code>. - The same scenario can occur if the <code>outMarket</code> does not have a provider, but the <code>inMarket</code> has a native token provider.
Recommendations	Consider wrapping or unwrapping <code>BNB/WBNB</code> before supplying as collateral.
Comments / Resolution	

Issue_43	LLTV check does not protect users from liquidations on partial migrations
Severity	Low
Description	During migration, users can provide an arbitrary value for <code>collateralAmount</code>, <code>borrowAmount</code>, and <code>borrowShares</code>. This allows customized position migrations. However, when a user migrates from one market to another, it is possible they are closer to liquidation on either market, depending on the collateral withdrawn/supplied and debt borrowed. The <code>migrateCommonMarketToFixedTermMarket</code> function implements an <code>inMarket.LLTV >= outMarket.LLTV</code> check to keep user positions safe from instant liquidations, but this does not really offer any protection on partial migrations.
Recommendations	Consider acknowledging this risk. As a precaution, provide users with a migration preview on the UI to avoid either position from being close to liquidation.
Comments / Resolution	

Issue_44	A malicious user can escape liquidations through migration to grief the liquidation BOT
Severity	Low
Description	A malicious user can frontrun escape liquidations using the function <code>migrateCommonMarketToFixedTermMarket</code> to grief the liquidation bot. For example: - <code>inMarket</code> LLTV = 80% and <code>outMarket</code> LLTV = 60%. - Position on <code>outMarket</code> is liquidatable. - The user migrates and escapes liquidation. The user can do this with a <code>minLoan</code> amount intentionally to grief the bot and select a fixed term with the lowest duration or lowest <code>APR</code> to leave the fixed term with minimal interest.
Recommendations	Consider implementing a health check in <code>migrateCommonMarketToFixedTermMarket</code> that ensures the user migrating is not liquidatable on the <code>outMarket</code> .
Comments / Resolution	

Issue_45	Unnecessary LLTV check prevents valid position migrations
Severity	Low
Description	Function <code>migrateCommonMarketToFixedTermMarket</code> implements the following check: <pre><i>require(inMarket.lltv >= outMarket.lltv, "in-market-lltv-too-low");</i></pre> This disallows migration to <code>inMarket</code> with LLTV less than <code>outMarket</code> LLTV, even when a user's position on <code>outMarket</code> is well within the <code>inMarket</code> LLTV. The check is implemented to protect users from being instantly liquidatable; however, this is unnecessary since Moolah would not allow creating an unhealthy position during migration. Additionally, as detailed in the issue titled - "LLTV check does not protect users from liquidations on partial migrations" - this check does not offer any protection.
Recommendations	Consider acknowledging this risk.
Comments / Resolution	

Issue_46	Anyone can DOS user migration by 1 wei frontrun repayments
Severity	Low
Description	During migration, the function <code>migrateCommonMarketToFixedTermMarket</code> takes a flash loan from Moolah to repay the debt to the <code>outMarket</code> . However, since anyone can frontrun a user's migration call to repay dust amounts on behalf of the user, full migrations can be DOSed.
Recommendations	Consider acknowledging this risk.
Comments / Resolution	

Issue_47	<code>PositionManager</code> migration DoSes during high-utilization periods because the flashloan needs idle liquidity at <code>Moolah</code>
Severity	Low
Description	<code>PositionManager.migrateCommonMarketToFixedTermMarket</code> uses <code>Moolah.flashLoan</code> to borrow the user's <code>outMarket</code> debt and repay it inside the callback. <code>Moolah.flashLoan()</code> does a plain <code>safeTransfer</code> of <code>assets</code> from Moolah's contract balance to the caller. If Moolah's idle balance of the loan token is less than the user's outstanding debt at the moment of migration, the transfer reverts, and the migration fails. There is no per-market reservation, so utilization across all markets sharing the loan token competes for the same pool. Large migrations during high-utilization windows are blocked until liquidity recovers, and a user with a large outstanding debt may not have a feasible migration path at all if their debt approaches the protocol-wide idle balance.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	

Issue_48	Migrations fail for Moolah markets with SmartProvider
Severity	Informational
Description	The SmartProvider contract has different function signatures for the withdrawCollateral and supplyCollateral functions compared to the ERC20Provider signatures used in the migration functionality. ERC20Provider: <pre>function supplyCollateral(MarketParams memory marketParams, uint256 assets, address onBehalf, bytes calldata data)</pre> SmartProvider: <pre>function supplyCollateral(MarketParams calldata marketParams, address onBehalf, uint256 amount0, uint256 amount1, uint256 minLpAmount)</pre> This leads to failed migrations for SmartProvider Moolah Markets.
Recommendations	Consider adding support for migrations involving SmartProvider moolah markets.
Comments / Resolution	

Issue_49	<code>PositionManager.initialize</code> accepts zero addresses for admin and manager
Severity	Informational
Description	<code>PositionManager</code> 's initializer grants <code>DEFAULT_ADMIN_ROLE</code> and <code>MANAGER</code> without validating non-zero addresses. A zero admin permanently freezes the proxy (no upgrade authority, no role administrator, no recovery) while a zero manager bricks migration (the entry point is gated by <code>MANAGER</code>).
Recommendations	Add explicit zero-address checks at the top of <code>initialize</code> .
Comments / Resolution	

Issue_50	Misleading partial migration comment in <code>onMoolahFlashLoan</code>
Severity	Informational
Description	The function <code>onMoolahFlashLoan</code> mentions that when <code>borrowShares</code> is greater than 0, share repayment will ensure exact full migration. However, the user can perform partial migration by specifying <code>borrowShares</code> partially. <i>When <code>borrowShares > 0</code>, repay by shares for exact full migration; when <code>borrowShares == 0</code>, assets is used (partial migration).</i>
Recommendations	Improvise the comment to mention partial migrations are possible when <code>borrowShares</code> is greater than 0.
Comments / Resolution	

Issue_51	No pause mechanism implemented
Severity	Informational
Description	The PositionManager allows users to migrate variable-rate borrow positions to fixed-term positions in a broker. This migration functionality, however, can never be paused in case of contract deprecation or an emergency. The same issue exists in the BrokerLiquidator contract, where important liquidation functionality cannot be paused.
Recommendations	Consider adding pausable functionality.
Comments / Resolution	

Issue_52	Migration can revert when inMarket or broker minLoan is greater than outMarket minLoan
Severity	Informational
Description	User migrations can revert due to inMarket moolah-level or broker-level minLoan requirements: - If a user has a position on outMarket with debt less than inMarket minLoan value, the migration can revert. - Even if the user satisfies the Moolah-level minLoan requirement, each broker position debt needs to be of at least minLoan amount, which could cause the revert.
Recommendations	Consider acknowledging this behaviour.
Comments / Resolution	

BrokerLiquidator

The `BrokerLiquidator` contract is an operational liquidation helper for broker-enabled Moolah markets. It is not the core lending protocol and does not own borrower accounting directly. Instead, it acts as an execution layer that routes liquidation calls into the relevant `LendingBroker`, receives seized collateral through the broker liquidation callback, optionally monetizes the collateral into the market loan token, and approves the broker to pull the repayment amount required to complete the liquidation.

The contract is configured through several manager-controlled whitelists. `marketIdToBroker` maps a Moolah market id to the broker contract that is allowed to liquidate that market through this helper. `brokerToMarketId` is the reverse mapping used to authenticate broker callbacks. `pairWhitelist` restricts low-level swap targets, `tokenWhitelist` restricts tokens that can be sold through `sellToken`, and `smartProviders` restricts smart collateral providers that can redeem LP-style collateral. The `BOT` role is the primary execution role for liquidations and token sales, while `MANAGER` controls configuration and asset withdrawals.

The helper supports three main liquidation styles. The first is a prefunded liquidation, where `BrokerLiquidator.liquidate()` calls the broker with either `seizedAssets` or `repaidShares` and expects the contract to already hold enough loan token to approve the broker during callback. The second is a flash-style collateral liquidation, where `flashLiquidate()` liquidates by `seizedAssets`, receives the seized collateral in the callback, swaps it into the loan token through a whitelisted pair/router, and approves the broker if the loan-token output is at least the broker-requested repayment. The third is smart-collateral liquidation, where seized LP/smart collateral is redeemed through a whitelisted smart provider and, depending on the mode, either redeemed after liquidation or redeemed and swapped during the broker callback.

Privileged Functions

- `withdrawERC20`
- `withdrawETH`
- `setTokenWhitelist`
- `setMarketToBroker`
- `batchSetMarketToBroker`
- `setPairWhitelist`
- `sellToken`
- `flashLiquidate`
- `liquidate`
- `liquidateSmartCollateral`
- `flashLiquidateSmartCollateral`
- `redeemSmartCollateral`
- `batchSetSmartProviders`

Core Invariants:

INV 1: `swapCollateral` and `swapSmartCollateral` cannot both be true

INV 2: Liquidations can only be carried out on whitelisted broker-market id pairs

INV 3: In flash liquidations, the proceeds from the collateral swap must cover the loan amount

INV 4: Swapping of collateral must be done via whitelisted pair contracts

Issue_53	Multiple risks in case of compromised governance roles
Severity	Governance
Description	There are multiple risks the contract currently has if the <code>DEFAULT_ADMIN_ROLE</code>, <code>MANAGER</code> or <code>BOT</code> role is compromised: 1. Privileged addresses can renounce the <code>DEFAULT_ADMIN_ROLE</code>, <code>MANAGER</code> and <code>BOT</code> roles to leave the contract in a state where: - Liquidations cannot occur for broker-level positions. - Contract cannot be upgraded - Tokens cannot be swapped or LP collateral cannot be redeemed - Loan and collateral tokens can remain locked in the contract 2. <code>MANAGER</code> can drain tokens from <code>BrokerLiquidator</code> using <code>withdrawERC20</code> and <code>withdrawETH</code>. 3. <code>MANAGER</code> can delete <code>marketIdToBroker/brokerToMarketId</code> mappings or set a new broker for the market even if there are existing broker positions active in the old broker. This can cause liquidations to fail for positions existing on the old broker contract. 4. <code>MANAGER</code> and <code>BOT</code> can collude to drain funds through <code>sellToken</code> if a malicious pair is registered. Even without <code>MANAGER</code> cooperation, the <code>BOT</code> can use <code>sellToken</code> with <code>amountOutMin = 0</code> plus <code>swapData</code> that encodes <code>recipient = BOT_address</code> to drain the contract through any whitelisted router.
Recommendations	Consider acknowledging this risk and using multi-signature wallets

	to prevent roles from being renounced or malicious operations from being executed instantly.
Comments / Resolution	

Issue_54	Pre-encoded swap data in <code>BrokerLiquidator.onMoolahLiquidate</code> can mismatch actual redeemed amounts, reverting smart-collateral liquidations
Severity	High
Description	The <code>swapSmartCollateral</code> path in <code>onMoolahLiquidate</code> redeems the LP via <code>SmartProvider.redeemLpCollateral</code> and then swaps the resulting <code>token0/token1</code> to the loan token. The redeemed <code>amount0</code> and <code>amount1</code> are determined at execution time (subject to stable-swap slippage), but the swap data (<code>swapToken0Data</code>, <code>swapToken1Data</code>) is encoded by the BOT off-chain ahead of time with a hardcoded input amount. When the actual amounts diverge from the encoded values, the swap either reverts (insufficient approval/balance) or under-routes the input, leaving residual tokens unswapped and producing <code>out < repaidAssets</code>, which trips the <code>NoProfit</code> revert. The native-token branch is doubly exposed: when <code>token0</code> or <code>token1</code> is BNB, the code passes <code>_value = arb.minToken0Amt</code> (or <code>minToken1Amt</code>), not the actual amount returned by redeem, so the swap is funded with the slippage floor rather than the redeemed amount, again producing under-routing and <code>NoProfit</code>. Two adjacent failure modes share the same <code>NoProfit</code> gate. First, pair-price manipulation: an attacker who can move the swap pair's spot price between BOT encoding and execution can push the swap's output below either the encoded <code>amountOutMin</code> (swap reverts with <code>SwapFailed</code>) or below <code>repaidAssets</code> (<code>NoProfit</code> reverts). Second, fundamentally unprofitable liquidations: when the underlying liquidation is in the dead zone (broker proportional fee exceeds LIF margin) or in the free-seize regime, the swap output

	cannot mathematically cover <code>repaidAssets</code> , so the swap path always trips <code>NoProfit</code> . A profitable liquidation can also be turned unprofitable by ordinary swap slippage. In all of these cases the swap path is unusable; the liquidator must fall back to the non-flash path (seize first, sell separately, accept inventory risk) or skip the user.
Recommendations	Consider encoding the swap input amount dynamically in <code>onMoolahLiquidate()</code> using the actual <code>amount0</code> and <code>amount1</code> returned by <code>redeemLpCollateral()</code>. The recommendation provided does not resolve the <code>NoProfit</code> revert scenario caused by slippage loss. We recommend using stable pools with low slippage and configuring the minimum output amount to ensure it is sufficient for repayment.
Comments / Resolution	

Issue_55	<code>sellToken</code> cannot sell native BNB residue
Severity	Low
Description	<code>sellToken</code> assumes the input token is ERC20: it calls <code>safeApprove(tokenIn, pair, amountIn)</code> and then <code>pair.call[swapData]</code> without value. The contract declares <code>receive() external payable {}</code> so it can hold native BNB, and the <code>SmartProvider</code> liquidation flow already forks on <code>BNB_ADDRESS</code> to use <code>pair.call{value: ...}[swapData]</code>. <code>sellToken</code> has no equivalent branch, so any native residue (from partial fills, malformed swap data, or direct sends) cannot be sold.
Recommendations	Add a BNB branch to <code>sellToken</code> .
Comments / Resolution	

Issue_56	Unverified <code>swapData</code> in <code>sellToken</code> allows non-whitelisted tokens and arbitrary input amount in swap path
Severity	Low
Description	The parameter <code>swapData</code> is unverified in the function <code>sellToken</code>, hence the swap path is also not verified and can include non-whitelisted tokens/pools in case of multi-hop swaps. Since the team plans to use 1Inch, PancakeSwap, etc., as registered swap routers, such untrusted tokens can be used. Another issue exists from the same root cause of unverified <code>swapData</code> where the actual input amount used differs from the <code>amountIn</code> parameter passed to the <code>sellToken</code> function. This would cause the below check to become redundant. <pre>require(actualAmountIn <= amountIn, ExceedAmount());</pre> Let us assume: - <code>swapData.amountIn</code> as 100, <code>amountIn</code> = 1000, contract balance is 2000. - If the pair uses 200 tokens [it has an approval of 1000 tokens], the check evaluates as $2000 - 1800 = 200$, which is less than <code>amountIn</code> of 1000 tokens. - This is clearly incorrect since the contract actually exceeded the real input amount of 100 tokens. Similarly, <code>swapCollateralData</code>, <code>swapToken0Data</code>, and <code>swapToken1Data</code> are unverified in the liquidation flow, hence can include untrusted tokens in the swap path, and a different input amount e.g., input amount in <code>swapCollateralData</code> can differ from <code>arb.seized</code>.
Recommendations	Consider acknowledging this issue and implementing trusted tokens verification off-chain. Alternatively, verify the swap data on-chain through a separate protocol signer address.
Comments / Resolution	

Issue_57	Liquidations can temporarily revert due to <code>checkPriceDiff</code> check in <code>StableSwapPool</code>
Severity	Low
Description	During liquidations, <code>StableSwapLPCollateral</code> tokens are redeemed through the <code>SmartProvider</code>, which calls the function <code>remove_liquidity</code> on the <code>StableSwapPool</code>. The function implements a <code>checkPriceDiff</code> check, which reverts if the price difference between the pool and the oracle exceeds the threshold. <pre>// Check if price differences exceed thresholds require((priceDiff0 * 1e18) <= (oraclePrices[0] * price0DiffThreshold), "Price difference for token0 exceeds threshold"); require((priceDiff1 * 1e18) <= (oraclePrices[1] * price1DiffThreshold), "Price difference for token1 exceeds threshold");</pre> This can temporarily disallow liquidations until the price difference check is manually disabled. <pre>if (!skipPriceDiff) checkPriceDiff(); // Check price diff before burning LP tokens</pre>
Recommendations	Consider acknowledging this risk and implementing measures to allow liquidations in case such a scenario arises.
Comments / Resolution	

Issue_58	LP pool reserve changes can revert smart-collateral liquidations at the redeem step
Severity	Low
Description	<code>SmartProvider.redeemLpCollateral</code> applies a slippage check against <code>arb.minToken0Amt</code> and <code>arb.minToken1Amt</code> , both pre-encoded by the BOT. Reserve drift between encoding and execution can push the redeem output below either floor, reverting before the swap step runs.
Recommendations	Encode min-out dynamically using a fresh on-chain quote or consider acknowledging this issue.
Comments / Resolution	

Issue_59	<code>liquidateSmartCollateral</code> requires both token pairs to be whitelisted, even when one side equals the loan token and is never swapped
Severity	Low
Description	<code>liquidateSmartCollateral</code> checks <code>pairWhitelist[token0Pair]</code> and <code>pairWhitelist[token1Pair]</code> unconditionally at the entrypoint, but the swap loop inside <code>onMoolahLiquidate</code> skips the swap entirely when <code>token0 == arb.loanToken</code> (or <code>token1 == loanToken</code>), since no swap is needed. The whitelist check is therefore enforced for a pair that will never be used. The BOT must either pre-whitelist a dummy address or pass a real whitelisted pair that will be ignored, both of which are workarounds. Furthermore, a 0 address cannot be passed in for the pair address here, due to the allowance reset at the end. <pre>if (token0 != BNB_ADDRESS) SafeTransferLib.safeApprove(token0, arb.token0Pair, 0); if (token1 != BNB_ADDRESS) SafeTransferLib.safeApprove(token1, arb.token1Pair, 0);</pre> This is because the openzeppelin ERC20 contract does not allow giving allowance to the 0 address, so the 0 allowance setting above will revert if either of the pair addresses is 0.
Recommendations	Make the pair-whitelist check conditional on whether the corresponding token actually requires swapping.
Comments / Resolution	

Issue_60	Liquidations can revert for token pairs that do not accept native tokens
Severity	Informational
Description	During liquidations, smart collateral token swaps can pass <code>msg.value</code> to the token pair contract if the redeemed LP collateral is native BNB. However, this will only work for Curve stable pools or pairs/routers that accept <code>msg.value</code>. Since multiple pairs can be whitelisted, if a pair does not accept <code>msg.value</code>, the call can revert, leading to failed liquidations.
Recommendations	Consider acknowledging this risk and using pairs that support native tokens when the input token is BNB.
Comments / Resolution	

Issue_61	Liquidations could temporarily revert due to paused pair contracts
Severity	Informational
Description	During liquidations, the <code>swapSmartCollateral</code> branch in <code>onMoolahLiquidate</code> redeems the LP collateral through the <code>SmartProvider</code> and swaps to the loan token using whitelisted pair contracts. However, if the pair is paused or deprecated with insufficient liquidity, the swap fails and reverts the liquidation call.
Recommendations	Consider acknowledging this risk and monitoring whitelisted pair contracts. Before adding pairs, consider validating or consulting to review integration risks.
Comments / Resolution	

Issue_62	BrokerLiquidator uses string-based require reverts instead of custom errors
Severity	Informational
Description	The BrokerLiquidator reverts with string messages for multiple require statements. Custom errors are cheaper at deploy and revert time, give callers a stable four-byte selector to match against, and keep error handling uniform across the codebase.
Recommendations	Replace each string-based require in BrokerLiquidator with a custom error
Comments / Resolution	

Issue_63	Consider using named mappings to improve code readability
Severity	Informational
Description	The BrokerLiquidator implements multiple mappings. The key-value fields of these mappings can be named to improve code readability. <pre>mapping(address => bool) public tokenWhitelist; mapping(address => bool) public pairWhitelist; mapping(bytes32 => address) public marketIdToBroker; mapping(address => bytes32) public brokerToMarketId; mapping(address => bool) public smartProviders;</pre>
Recommendations	Consider naming the key-value fields of the mappings or acknowledge.
Comments / Resolution	

Issue_64	Consider removing the unused <code>_lastRepaidAssets</code> variable
Severity	Informational
Description	The <code>_lastRepaidAssets</code> variable does not have any utility in the <code>BrokerLiquidator</code> contract. <pre> /// @dev transient storage for repaidAssets from onMoolahLiquidate callback uint256 internal _lastRepaidAssets; </pre>
Recommendations	Consider removing the variable or documenting the reason for the variable implementation.
Comments / Resolution	

Issue_65	<code>_lastRepaidAssets</code> is not transient
Severity	Informational
Description	The <code>_lastRepaidAssets</code> variable has a comment indicating that it is transient. <pre> /// @dev transient storage for repaidAssets from onMoolahLiquidate callback uint256 internal _lastRepaidAssets; </pre> However, it is defined as a normal storage slot.
Recommendations	Consider removing it if unused or changing it to transient.
Comments / Resolution	

Issue_66	Possible risk with hardcoded <code>seizedAssets</code> in callback data
Severity	Informational
Description	Note: This issue cannot arise in the codebase currently; however, the behaviour is being noted for future reference in case of contract upgrades or integrations. The <code>seized</code> variable in the abi-encoded <code>MoolahLiquidateData</code> struct is fixed at the start of any of the <code>BrokerLiquidator</code> liquidation operations i.e., <code>flashLiquidate</code>, <code>liquidate</code>, <code>liquidateSmartCollateral</code>, and <code>flashLiquidateSmartCollateral</code>. However, the actual <code>seizedAssets</code> can actually be overwritten by Moolah if <code>repaidShares</code> is being used. <pre> seizedAssets = repaidShares .toAssetsDown[market[id].totalBorrowAssets, market[id].totalBorrowShares] .wMulDown[liquidationIncentiveFactor] .mulDivDown[ORACLE_PRICE_SCALE, collateralPrice]; </pre> In that case, the fixed seized value in the struct would be incorrect, leading to failed liquidations in <code>BrokerLiquidator.onMoolahLiquidate</code>, since seized collateral assets to loan token swaps revert (due to insufficient assets or approval). Additionally, Moolah overwriting <code>seizedAssets</code> would make it difficult to predict input amount values in <code>swapCollateralData</code>, <code>swapToken0Data</code>, and <code>swapToken1Data</code>.
Recommendations	Consider acknowledging this risk.
Comments / Resolution	

Issue_67	Improvise error message in normal swap collateral branch
Severity	Informational
Description	The <code>swapCollateral</code> branch in the <code>onMoolahLiquidate</code> function adds the require check to ensure that smart collateral cannot be swapped in the normal swap collateral branch. However the error message is misleading and should be corrected to “only swap collateral” since smart collateral cannot be swapped in this branch. <pre> if (arb.swapCollateral) { require(!arb.swapSmartCollateral, "only swap collateral or smart collateral"); } </pre>
Recommendations	Improvise the error message to “only swap collateral”.
Comments / Resolution	

Moolah

The **Moolah** vault is the underlying lending protocol of the system. This vault has been covered in previous audits, and only the changes and interaction with the new modules are in scope of this current audit. Since the lending broker functionality is built on top of Moolah, this contract is included here for completeness and issues related to this interactions are listed.

Issue_68	Blacklisting the curated vault freezes broker repays and liquidations
Severity	Governance
Description	Moolah.supply reverts with BLACKLISTED whenever onBehalf is in vaultBlacklist . The relayer flushes broker interest by calling MOOLAH.supply(..., vault, "") with no try/catch, the broker's _supplyToMoolahVault() has no fallback, and the user's repayment or liquidation reverts entirely. Once the curated vault is blacklisted (a single MANAGER call to setVaultBlacklist), every interest-crossing-threshold call through the relayer fails. Borrowers cannot repay their interest portion, liquidators cannot clear unhealthy positions, convert-to-fixed and any other flow that routes interest through the relayer fails identically.
Recommendations	Consider wrapping the inner MOOLAH.supply call in try/catch so a blacklisted-vault state does not cause the transaction to revert. Furthermore, consider adding an admin-restricted setVault to redirect future flushes to a non-blacklisted vault, and a rescueTokens() function to evacuate trapped residual.
Comments / Resolution	

Issue_69	Malicious/compromised MANAGER can use the function liquidateBrokerPosition to liquidate all users at any time
Severity	Governance
Description	The liquidateBrokerPosition function allows a registered broker to call it. If a MANAGER registers a malicious broker address, it can be used to liquidate any user at any time. Since the function does not implement important validation, such as health checks similar to the Moolah.liquidate function to ensure the user is liquidatable, the call succeeds, erasing all of the user's supplied assets and debt.
Recommendations	Consider removing the function if it is not intended to be used or documenting its purpose. If the function will not be removed, implement important health checks to protect users.
Comments / Resolution	

Issue_70	<code>setMarketBroker</code> can be added to a non-empty market, breaking the proportional <code>interestToBroker</code> fee
Severity	Governance
Description	<code>Moolah.setMarketBroker(id, broker, true)</code> has no check that <code>market[id].totalBorrowShares == 0</code>. A market created without a broker (or with the broker temporarily removed) can have direct Moolah borrows that persist after the broker is registered. The broker's <code>getTotalDebt</code> does not see those positions, but Moolah's <code>getDebtAtMoolah</code> does. The proportional fee formula in <code>onMoolahLiquidate()</code> then divides by an inflated <code>debtAtMoolah</code>, structurally undercharging the broker. When the legacy direct portion is large enough that <code>totalDebtAtBroker < debtAtMoolah</code>, the formula floors at 0, and the broker collects nothing. A separate consequence of the same gap is that direct calls to <code>Moolah.borrow</code> with shares specified (<code>assets = 0</code>) during the no-broker window can record share counts at Moolah that were never matched by an actual asset borrow. The resulting TBA / TBS ratio drift propagates to every subsequent broker-routed borrow and repay: legitimate borrows mint inflated shares, and repays compute share-burn quantities that exceed the user's actual share balance, reverting via underflow. The empty-market precondition on <code>setMarketBroker</code> also closes this angle. In the case of liquidations using <code>repaidShares</code>, it is possible that due to the inflated <code>borrowShares</code>, we reach a state where borrowed shares are greater than zero while borrowed assets are zero for the borrower. This would lead to a situation where the inflated shares deflate the interest on the <code>LendingBroker</code> over time.
Recommendations	In <code>setMarketBroker</code> , require <code>market[id].totalBorrowShares == 0</code> on addition and requires no broker-tracked positions on removal.
Comments / Resolution	

Issue_71	Collateral price of zero due to excessive interest breaks liquidation logic
Severity	Medium
Description	When the collateral price is calculated, the true collateral price is fetched from the oracle first. Afterward, it is discounted by the interest accrued at the broker level. When too much interest accrues, such that the deduction is greater than or equal to the collateral price, a price of 0 is returned instead. However, this leads to problems in the liquidation flow: When seizedAssets > 0 and collateralPrice == 0, the repaidShares calculation results in 0. As a result, a seizedAssets amount of collateral is transferred to the liquidator without any shares being repaid. When repaidShares > 0 and collateralPrice == 0, the seizedAssets calculation reverts due to division by 0. This means that liquidations do not work in this case.
Recommendations	Implement special liquidation logic when the collateralPrice is 0 due to excessive interest (e.g., seize all collateral and write off debt).
Comments / Resolution	

Issue_72	Vault blacklist check is not implemented on withdraw
Severity	Low
Description	Currently, the vaultBlacklist check is only implemented on the supply function. This means any user blacklisted after having supplied assets and earning interest on them can withdraw those assets.
Recommendations	Consider applying the blacklist check on withdraw .
Comments / Resolution	

Issue_73	Implementation of the unused liquidateBrokerPosition function should be removed or documented
Severity	Low
Description	Function liquidateBrokerPosition is never used since the broker never calls it. Additionally, the function does not implement important validation, such as health checks similar to the Moolah.liquidate function, to ensure the user is liquidatable and if the broker is in the liquidationWhitelist .
Recommendations	Consider removing the function or documenting its purpose.
Comments / Resolution	